API > generic > nb

base nb numba

base module

Generic Numba-compiled functions for base operations.

---

bfill_1d_nb function

```
bfill_1d_nb(
    arr
)
```

Fill NaNs by propagating first valid observation backward.

Numba equivalent to `pd.Series(arr).fillna(method='bfill')`.

Warning

This operation looks ahead.

---

bfill_nb function

```
bfill_nb(
    arr
)
```

2-dim version of bfill_1d_nb.

---

bshift_1d_nb function

```
bshift_1d_nb(
    arr,
    n=1,
    fill_value=nan
)
```

Shift backward by `n` positions.

Numba equivalent to `pd.Series(arr).shift(-n)`.

Warning

This operation looks ahead.

---

bshift_nb function

```
bshift_nb(
    arr,
    n=1,
    fill_value=nan
)
```

2-dim version of **bshift_1d_nb**.

---

crossed_above_1d_nb function

```
crossed_above_1d_nb(
    arr1,
    arr2,
    wait=0,
    dropna=False
)
```

Get the crossover of the first array going above the second array.

If `dropna` is True, produces the same results as if all rows with at least one NaN were dropped.

---

crossed_above_nb function

```
crossed_above_nb(
    arr1,
    arr2,
    wait=0,
    dropna=False
)
```

2-dim version of **crossed_above_1d_nb**.

---

crossed_below_1d_nb function

```
crossed_below_1d_nb(
    arr1,
    arr2,
    wait=0,
    dropna=False
)
```

Get the crossover of the first array going below the second array.

If `dropna` is True, produces the same results as if all rows with at least one NaN were dropped.

---

crossed_below_nb function

```
crossed_below_nb(
```

```
    arr1,
    arr2,
    wait=0,
    dropna=False
)
```

2-dim version of **crossed_below_1d_nb**.

---

demean_nb function

```
demean_nb(
    arr,
    group_map
)
```

Demean each value within its group.

---

diff_1d_nb function

```
diff_1d_nb(
    arr,
    n=1
)
```

Compute the 1-th discrete difference.

Numba equivalent to `pd.Series(arr).diff()`.

---

diff_nb function

```
diff_nb(
    arr,
    n=1
)
```

2-dim version of **diff_1d_nb**.

---

fbfill_nb function

```
fbfill_nb(
    arr
)
```

Forward and backward fill NaN values.

> **Note**
>
> If there are no NaN (or any) values, will return `arr`.

---

ffill_1d_nb function

```
ffill_1d_nb(
    arr
)
```

Fill NaNs by propagating last valid observation forward.

Numba equivalent to `pd.Series(arr).fillna(method='ffill')`.

---

ffill_nb function

```
ffill_nb(
```

```
    arr
)
```

2-dim version of **ffill_1d_nb**.

---

fillna_1d_nb function

```
fillna_1d_nb(
    arr,
    value
)
```

Replace NaNs with value.

Numba equivalent to `pd.Series(arr).fillna(value)`.

---

fillna_nb function

```
fillna_nb(
    arr,
    value
)
```

2-dim version of **fillna_1d_nb**.

---

fir_filter_1d_nb function

```
fir_filter_1d_nb(
    b,
    x
)
```

Filter data along one-dimension with an FIR filter.

first_valid_index_1d_nb function

```
first_valid_index_1d_nb(
    arr,
    check_inf=True
)
```

Get the index of the first valid value.

---

first_valid_index_nb function

```
first_valid_index_nb(
    arr,
    check_inf=True
)
```

2-dim version of **first_valid_index_1d_nb**.

---

fshift_1d_nb function

```
fshift_1d_nb(
    arr,
    n=1,
    fill_value=nan
)
```

Shift forward by `n` positions.

Numba equivalent to `pd.Series(arr).shift(n)`.

---

fshift_nb function

```
fshift_nb(
    arr,
    n=1,
    fill_value=nan
)
```

2-dim version of **fshift_1d_nb**.

---

last_valid_index_1d_nb function

```
last_valid_index_1d_nb(
    arr,
    check_inf=True
)
```

Get the index of the last valid value.

---

last_valid_index_nb function

```
last_valid_index_nb(
    arr,
    check_inf=True
)
```

2-dim version of **last_valid_index_1d_nb**.

---

nancnt_1d_nb function

```
nancnt_1d_nb(
    arr
)
```

Compute count while ignoring NaNs and not allocating any arrays.

nancnt_nb function

```
nancnt_nb(
    arr
)
```

2-dim version of **nancnt_1d_nb**.

---

nancorr_1d_nb function

```
nancorr_1d_nb(
    arr1,
    arr2
)
```

Numba equivalent of `np.corrcoef` that ignores NaN values.

Numerically stable.

---

nancorr_nb function

```
nancorr_nb(
    arr1,
    arr2
)
```

2-dim version of **nancorr_1d_nb**.

---

nancov_1d_nb function

```
nancov_1d_nb(
    arr1,
    arr2,
    ddof=0
)
```

Numba equivalent of `np.cov` that ignores NaN values.

---

nancov_nb function

```
nancov_nb(
    arr1,
    arr2,
    ddof=0
)
```

2-dim version of **nancov_1d_nb**.

---

nancumprod_nb function

```
nancumprod_nb(
    arr
)
```

Numba equivalent of `np.nancumprod` along axis 0.

---

nancumsum_nb function

```
nancumsum_nb(
    arr
)
```

Numba equivalent of `np.nancumsum` along axis 0.

nanmax_nb function

```
nanmax_nb(
    arr
)
```

Numba equivalent of `np.nanmax` along axis 0.

nanmean_nb function

```
nanmean_nb(
    arr
)
```

Numba equivalent of `np.nanmean` along axis 0.

nanmedian_nb function

```
nanmedian_nb(
    arr
)
```

Numba equivalent of `np.nanmedian` along axis 0.

nanmin_nb function

```
nanmin_nb(
    arr
)
```

Numba equivalent of `np.nanmin` along axis 0.

---

nanpartition_mean_noarr_1d_nb function

```
nanpartition_mean_noarr_1d_nb(
    arr,
    q
)
```

Average of `np.partition` that ignores NaN values and does not allocate any arrays.

Note

Has worst case time complexity of O(N^2), which makes it much slower than `np.partition`, but still faster if used in rolling calculations, especially for `q` near 0.

---

nanpercentile_noarr_1d_nb function

```
nanpercentile_noarr_1d_nb(
    arr,
    q
)
```

Numba equivalent of `np.nanpercentile` that does not allocate any arrays.

Note

Has worst case time complexity of O(N^2), which makes it much slower than `np.nanpercentile`, but still faster if used in rolling calculations, especially for `q` near 0 and 100.

---

nanprod_nb function

```
nanprod_nb(
    arr
)
```

Numba equivalent of `np.nanprod` along axis 0.

---

nanstd_1d_nb function

```
nanstd_1d_nb(
    arr,
    ddof=0
)
```

Numba equivalent of `np.nanstd`.

---

nanstd_nb function

```
nanstd_nb(
    arr,
    ddof=0
)
```

2-dim version of **nanstd_1d_nb**.

---

nansum_nb function

```
nansum_nb(
    arr
)
```

Numba equivalent of `np.nansum` along axis 0.

---

nanvar_1d_nb function

```
nanvar_1d_nb(
    arr,
    ddof=0
)
```

Numba equivalent of `np.nanvar` that does not allocate any arrays.

---

pct_change_1d_nb function

```
pct_change_1d_nb(
    arr,
    n=1
)
```

Compute the percentage change.

Numba equivalent to `pd.Series(arr).pct_change()`.

---

pct_change_nb function

```
pct_change_nb(
    arr,
    n=1
)
```

2-dim version of **pct_change_1d_nb**.

---

polyfit_1d_nb function

```
polyfit_1d_nb(
    x,
    y,
    deg,
    stabilize=False
)
```

Compute the least squares polynomial fit.

---

rank_1d_nb function

```
rank_1d_nb(
    arr,
    argsorted=None,
    pct=False
)
```

Compute numerical data ranks.

Numba equivalent to `pd.Series(arr).rank(pct=pct)`.

---

rank_nb function

```
rank_nb(
    arr,
    argsorted=None,
    pct=False
)
```

2-dim version of **rank_1d_nb**.

---

realign_1d_nb function

```
realign_1d_nb(
    arr,
    source_index,
    target_index,
    source_freq=None,
    target_freq=None,
    source_rbound=False,
    target_rbound=None,
    nan_value=nan,
    ffill=True
)
```

Get the latest in `arr` at each index in `target_index` based on `source_index`.

If `source_rbound` is True, then each element in `source_index` is effectively located at the right bound, which is the frequency or the next element (excluding) if the frequency is None. The same for `target_rbound` and `target_index`.

Note

Both index arrays must be increasing. Repeating values are allowed.

If `arr` contains bar data, both indexes must represent the opening time.

---

realign_nb function

```
realign_nb(
    arr,
    source_index,
    target_index,
    source_freq=None,
    target_freq=None,
    source_rbound=False,
    target_rbound=False,
    nan_value=nan,
    ffill=True
```

```
)
```

2-dim version of **realign_1d_nb**.

---

repartition_nb function

```
repartition_nb(
    arr,
    counts
)
```

Repartition a 2-dimensional array into a 1-dimensional by removing empty elements.

---

select_indices_1d_nb function

```
select_indices_1d_nb(
    arr,
    indices,
    fill_value
)
```

Set each element to a value by boolean mask.

---

select_indices_nb function

```
select_indices_nb(
    arr,
    indices,
    fill_value
)
```

2-dim version of **select_indices_1d_nb**.

set_by_mask_1d_nb function

```
set_by_mask_1d_nb(
    arr,
    mask,
    value
)
```

Set each element to a value by boolean mask.

---

set_by_mask_mult_1d_nb function

```
set_by_mask_mult_1d_nb(
    arr,
    mask,
    values
)
```

Set each element in one array to the corresponding element in another by boolean mask.

`values` must be of the same shape as in the array.

---

set_by_mask_mult_nb function

```
set_by_mask_mult_nb(
    arr,
    mask,
    values
)
```

2-dim version of **set_by_mask_mult_1d_nb**.

---

set_by_mask_nb function

```
set_by_mask_nb(
    arr,
    mask,
    value
)
```

2-dim version of **set_by_mask_1d_nb**.

---

shuffle_1d_nb function

```
shuffle_1d_nb(
    arr,
    seed=None
)
```

Shuffle each column in the array.

Specify `seed` to make output deterministic.

---

shuffle_nb function

```
shuffle_nb(
    arr,
    seed=None
)
```

2-dim version of **shuffle_1d_nb**.

---

to_renko_1d_nb function

```
to_renko_1d_nb(
```

```
    arr,
    brick_size,
    relative=False,
    start_value=None,
    max_out_len=None
)
```

Convert to Renko format.

---

to_renko_ohlc_1d_nb function

```
to_renko_ohlc_1d_nb(
    arr,
    brick_size,
    relative=False,
    start_value=None,
    max_out_len=None
)
```

Convert to Renko OHLC format.

---

value_counts_1d_nb function

```
value_counts_1d_nb(
    codes,
    n_uniques
)
```

Compute value counts.

---

value_counts_nb function

```
value_counts_nb(
    codes,
```

```
    n_uniques,
    group_map
)
```

Compute value counts per column/group.

---

value_counts_per_row_nb function

```
value_counts_per_row_nb(
    codes,
    n_uniques
)
```

Compute value counts per row.